

Study Report: Python Developer - Career Path & Core Skills

Written by Gagan Pasupuleti

Family:	Python Developer
Level:	Beginner to Advanced
Tracks:	Beginner, Intermediate, Advanced
Chapters:	18
Source:	Official docs, free libraries, and industry-standard references (see sources and books)

Official sources and free libraries

- <https://docs.python.org/3/>
- <https://fastapi.tiangolo.com/>
- <https://docs.djangoproject.com/en/stable/>
- <https://docs.pytest.org/>
- <https://docs.pydantic.dev/latest/>
- <https://packaging.python.org/>

Summary

Full-stack Python engineering with Python 3.12+, FastAPI, Django, pytest, async I/O, SQLAlchemy, Docker, and modern packaging. Covers scripting fluency through async APIs, test automation, and production deployment patterns used in startups and data-driven teams.

Skill stack by level

Beginner

- Python 3.12 syntax, types, and control flow
- Virtual environments (venv, uv, pip)
- Functions, modules, and package structure
- File I/O, JSON, and pathlib
- List/dict comprehensions and generators
- Basic debugging with pdb and IDE breakpoints
- Git and collaborative workflow

Intermediate

- FastAPI and Pydantic v2 request/response models
- Django ORM, admin, and REST with DRF
- SQLAlchemy 2.0 async sessions

- pytest fixtures, parametrize, and coverage
- asyncio and httpx for concurrent I/O
- Docker and docker-compose for local stacks
- Environment configuration with pydantic-settings
- CI with GitHub Actions and ruff/mypy linting

Advanced

- Background tasks (Celery, ARQ, Dramatiq)
- OAuth2 and JWT auth patterns
- Observability (structlog, OpenTelemetry)
- Performance profiling (py-spy, scalene)
- Multi-stage Docker and cloud deployment
- Database migrations (Alembic)
- API versioning and rate limiting

Recommended books

1. Automate the Boring Stuff with Python - Al Sweigart

Level: Beginner | Practical scripts for files, Excel, web scraping, and automation.

CodeQuest:

Book_Reports/10-comprehensive-and-projects/study-report-automate-the-boring-stuff-with-python-practical-programmi/report.json

2. Python Crash Course - Eric Matthes

Level: Beginner | Hands-on basics, data viz, and a Django web app project.

CodeQuest:

Book_Reports/01-python-beginners-core/study-report-coding-for-beginners-using-python-a-hands-on-project-base/report.json

3. Effective Python - Brett Slatkin

Level: Intermediate | 90 patterns for readable, efficient Python 3 code.

4. Architecture Patterns with Python - Harry Percival & Bob Gregory

Level: Intermediate | Repository pattern, domain-driven design, and TDD in Python.

5. Fluent Python - Luciano Ramalho

Level: Advanced | Deep dive into data model, descriptors, async, and metaprogramming.

End-to-end projects

Project 1: CLI Task Manager with SQLite

Beginner | 1-2 weeks - Command-line todo app with SQLite persistence, argparse, and pytest coverage.

- Schema: schema.sql
- CLI: Working CLI
- Tests: 90% coverage
- Polish: pip installable package

Project 2: FastAPI REST + PostgreSQL Blog API

Intermediate | 3-4 weeks - Async FastAPI blog with SQLAlchemy 2.0, JWT auth, Alembic migrations, and Docker deployment.

- Models: Migrated schema

- Auth: Auth Postman tests
- API: /docs working
- Deploy: Live URL

Project 3: Production ML Inference Service

Advanced | 4-6 weeks - FastAPI service serving a scikit-learn model with batch prediction, caching, OpenTelemetry, and k6 load tests.

- Model: model.pkl v1
- API: Latency benchmarks
- Cache: Cache hit metrics
- Load Test: k6 HTML report

Chapters

Track: Beginner

Chapter 1: Python 3.12 Developer Environment [Beginner]

Chapter focus: Python 3.12 Developer Environment** *(Level: Beginner)

Python 3.12 improves error messages, adds per-interpreter GIL options, and refines typing. Use venv or uv to isolate project dependencies from system Python. pyproject.toml is the modern standard for metadata, dependencies, and tool configuration. Configure your editor with Ruff for lint/format and mypy for static type checking early.

Code Reference:

```
python -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
pip install -r requirements.txt
python --version
```

What it shows: Creates isolated environment and verifies Python version before installing packages.

Topic guide

What it actually is

A Python developer environment includes interpreter, virtual env, package manager, and linter.

When to use it

Set up on day one of any new project; never install packages globally for application work.

Where to use it

Local development, tutorials, CodeQuest exercises, and open-source contributions.

Real use example

A new hire runs one setup script and executes pytest successfully within ten minutes.

Chapter 2: Variables, Types, and Control Flow [Beginner]

Chapter focus: Variables, Types, and Control Flow** *(Level: Beginner)

Python uses dynamic typing with optional static hints via type annotations. Indentation defines blocks-consistency matters for readability and team reviews. if/elif/else, for, and while loops cover most control flow; match (3.10+) handles structural patterns. f-strings are the preferred way to format strings with embedded expressions.

Code Reference:

```
name: str = "Ada"
age: int = 28
if age >= 18:
    print(f"{name} is an adult")
```

What it shows: Type hints document intent; f-string embeds variable values in output.

Topic guide

What it actually is

Core syntax expresses logic, data, and branching in readable Python code.

When to use it

Master before building web apps-every framework builds on these primitives.

Where to use it

Scripts, notebooks, API handlers, and test functions.

Real use example

An onboarding script reads config.json and prints personalized welcome messages.

Chapter 3: Functions, Modules, and Packages [Beginner]

Chapter focus: Functions, Modules, and Packages** *(Level: Beginner)

Functions group reusable logic; *args and **kwargs accept variable arguments. Modules are .py files; packages are directories with __init__.py (or namespace packages). Import only what you need to keep namespaces clean and startup fast. if __name__ == '__main__' guard allows dual use as script and importable module.

Code Reference:

```
def normalize_email(address: str) -> str:
    return address.strip().lower()

if __name__ == "__main__":
    print(normalize_email(" User@Example.COM "))
```

What it shows: Function returns cleaned email; main guard runs demo only when executed directly.

Topic guide

What it actually is

Modules organize code into reusable units with clear public APIs.

When to use it

Split growing scripts into modules when files exceed ~200 lines or responsibilities multiply.

Where to use it

src/ layouts, FastAPI routers, Django apps, and shared utility libraries.

Real use example

utils/validators.py exports email checks imported by both CLI and API layers.

Chapter 4: Data Structures and Comprehensions [Beginner]**Chapter focus: Data Structures and Comprehensions** *(Level: Beginner)**

Lists, dicts, sets, and tuples are the primary built-in collections. Comprehensions build collections concisely: `[x*2 for x in range(5)]`. `dict.get` avoids `KeyError`; `defaultdict` and `Counter` from collections solve common patterns. Choose the right structure: list for order, set for uniqueness, dict for lookups.

Code Reference:

```
users = [{"id": 1, "role": "admin"}, {"id": 2, "role": "viewer"}]
admins = [u["id"] for u in users if u["role"] == "admin"]
```

What it shows: List comprehension filters admins in one readable expression.

Topic guide**What it actually is**

Collections store and organize data; comprehensions transform them declaratively.

When to use it

Use comprehensions for small transforms; use loops when logic branches heavily.

Where to use it

ETL prep, API response shaping, config parsing, and test data builders.

Real use example

A report generator aggregates sales by region using `dict.setdefault` in ten lines.

Chapter 5: File I/O, JSON, and pathlib [Beginner]**Chapter focus: File I/O, JSON, and pathlib** *(Level: Beginner)**

`pathlib.Path` provides object-oriented file system paths cross-platform. `open()` with context managers ensures files close even on exceptions. `json` module serializes Python dicts to JSON for APIs and config files. Never hardcode paths-use `Path(__file__).parent` to anchor relative locations.

Code Reference:

```
from pathlib import Path
import json

config_path = Path(__file__).parent / "config.json"
config = json.loads(config_path.read_text(encoding="utf-8"))
```

What it shows: Path joins directories portably; read_text loads JSON config safely with encoding.

Topic guide**What it actually is**

File I/O reads and writes persistent data; JSON is the lingua franca for config and APIs.

When to use it

Use pathlib and json for configs, exports, logs, and batch imports.

Where to use it

CLI tools, data pipelines, Django settings loaders, and test fixtures.

Real use example

A deployment script reads config.json for database URL instead of hardcoding secrets.

Chapter 6: Intro to pytest and Testable Code [Beginner]**Chapter focus: Intro to pytest and Testable Code** (Level: Beginner)**

pytest discovers test_ functions and uses assert directly-no boilerplate classes. Fixtures inject setup/teardown dependencies like database sessions or temp directories. Arrange-Act-Assert structure keeps tests readable and focused on one behavior. Write tests alongside features; untested code is legacy the moment it merges.

Code Reference:

```
def test_discount_applies_percent():
    price = apply_discount(100.0, percent=10)
    assert price == 90.0
```

What it shows: Single assertion verifies discount math; pytest reports clear failures on regression.

Topic guide**What it actually is**

pytest is Python's dominant testing framework for unit and integration tests.

When to use it

Add tests for every bug fix and new function before marking tickets done.

Where to use it

All professional Python repos, CI pipelines, and CodeQuest project submissions.

Real use example

Teacher review passes because pytest coverage proves edge cases were handled.

Track: Intermediate

Chapter 7: FastAPI and Pydantic v2 Models [Intermediate]**Chapter focus: FastAPI and Pydantic v2 Models** *(Level: Intermediate)**

FastAPI builds on Starlette and Pydantic for high-performance typed APIs. Pydantic v2 uses `model_validate` and computed fields with Rust-core speed. Dependency injection via `Depends()` shares DB sessions, auth, and settings cleanly. Automatic OpenAPI generation documents endpoints for frontend and QA teams.

Code Reference:

```
from pydantic import BaseModel, Field

class CreateItem(BaseModel):
    name: str = Field(min_length=1, max_length=120)
    price: float = Field(gt=0)

@app.post("/items", response_model=ItemOut)
async def create_item(payload: CreateItem, db: DbSession):
    return await service.create(db, payload)
```

What it shows: Pydantic validates payload before handler runs; response_model strips internal fields.

Topic guide**What it actually is**

FastAPI is a modern async web framework for building APIs with automatic validation and docs.

When to use it

Choose FastAPI for new microservices, ML model servers, and high-throughput JSON APIs.

Where to use it

Startups, internal tools, mobile backends, and ML inference gateways.

Real use example

Frontend codegen from `/openapi.json` produces TypeScript clients matching backend types exactly.

Chapter 8: Django ORM and Admin Productivity [Intermediate]**Chapter focus: Django ORM and Admin Productivity** *(Level: Intermediate)**

Django's ORM maps models to tables with migrations tracking schema evolution. The admin site gives non-developers CRUD UI for free with customizable `ModelAdmin` classes. `QuerySet` API chains `filter`, `exclude`, `select_related`, and `prefetch_related` for efficient queries. Use Django for

content-heavy apps, internal dashboards, and rapid MVP delivery.

Code Reference:

```
class Article(models.Model):
    title = models.CharField(max_length=200)
    published = models.DateTimeField(auto_now_add=True)

recent = Article.objects.filter(title__icontains="python").order_by("-published")[:10]
```

What it shows: CharField maps to VARCHAR; slice limits queryset evaluation to ten rows.

Topic guide

What it actually is

Django is a batteries-included web framework with ORM, auth, admin, and templating.

When to use it

Use when you need admin UI, user auth, and relational models without assembling pieces manually.

Where to use it

CMS platforms, internal ops tools, e-commerce backends, and multi-app SaaS cores.

Real use example

Support staff update product descriptions via /admin without developer involvement.

Chapter 9: asyncio and Concurrent I/O [Intermediate]

Chapter focus: asyncio and Concurrent I/O** *(Level: Intermediate)

asyncio event loop schedules coroutines cooperatively on a single thread. await yields control during I/O waits, allowing thousands of concurrent connections. Use httpx.AsyncClient and async database drivers-never block the loop with time.sleep. asyncio.gather runs independent coroutines concurrently and collects results.

Code Reference:

```
async def fetch_all(urls: list[str]) -> list[str]:
    async with httpx.AsyncClient(timeout=10) as client:
        tasks = [client.get(url) for url in urls]
        responses = await asyncio.gather(*tasks)
        return [r.text for r in responses]
```

What it shows: AsyncClient reuses connections; gather fetches all URLs concurrently.

Topic guide

What it actually is

asyncio enables high-concurrency I/O-bound Python without multithreading complexity.

When to use it

Use async endpoints when calling external APIs, websockets, or async DB drivers.

Where to use it

FastAPI services, chat servers, scrapers, and fan-out aggregation APIs.

Real use example

A pricing API fetches ten vendor quotes in parallel, cutting latency from 5s to 600ms.

Chapter 10: SQLAlchemy 2.0 Async Sessions [Intermediate]**Chapter focus: SQLAlchemy 2.0 Async Sessions** *(Level: Intermediate)**

SQLAlchemy 2.0 unified Core and ORM with `select()` style queries. `AsyncSession` pairs with `asyncpg` for non-blocking PostgreSQL access in FastAPI. Explicit session scope per request prevents connection leaks in long-running apps. Alembic autogenerates migrations from model metadata changes.

Code Reference:

```
async with async_session() as session:
    result = await session.execute(select(User).where(User.active.is_(True)))
    users = result.scalars().all()
```

What it shows: Async context manager commits/rolls back; `select()` replaces legacy query API.

Topic guide**What it actually is**

SQLAlchemy is Python's powerful SQL toolkit and ORM with sync and async modes.

When to use it

Use with FastAPI or standalone services needing fine-grained SQL control.

Where to use it

PostgreSQL-backed APIs, analytics ETL, and multi-database integrations.

Real use example

Alembic migration adds index on `users.email`; deploy applies it before new code rolls out.

Chapter 11: pytest Fixtures, Mocking, and Coverage [Intermediate]**Chapter focus: pytest Fixtures, Mocking, and Coverage** *(Level: Intermediate)**

Fixtures with `scope='module'` share expensive setup across tests in a file. `monkeypatch` and `unittest.mock` patch external HTTP calls and environment variables. `pytest-asyncio` marks async test functions for automatic loop management. `Coverage.py` highlights untested branches-aim for meaningful coverage, not 100% busywork.

Code Reference:

```
@pytest.fixture
def client(db_session):
    app.dependency_overrides[get_db] = lambda: db_session
    yield TestClient(app)
```

```
app.dependency_overrides.clear()
```

What it shows: Override DB dependency so tests hit in-memory SQLite without network.

Topic guide

What it actually is

pytest fixtures and mocks isolate units under test from external systems.

When to use it

Mock third-party APIs; use real DB containers for integration tests.

Where to use it

Every Python CI pipeline and CodeQuest project grading rubric.

Real use example

CI fails when coverage drops below 85% on new pull requests.

Chapter 12: Docker for Python Services [Intermediate]

Chapter focus: Docker for Python Services** *(Level: Intermediate)

Docker images bundle Python, dependencies, and app code for reproducible runs. Multi-stage builds use a builder stage with compile deps and slim runtime stage. docker-compose orchestrates API + Postgres + Redis for local parity with production. Use .dockerignore to exclude .venv, __pycache__, and secrets from build context.

Code Reference:

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

What it shows: Slim base reduces image size; uvicorn serves FastAPI on container port 8000.

Topic guide

What it actually is

Containers package Python apps with exact dependency versions for any environment.

When to use it

Containerize before deploying to cloud platforms or sharing with teammates.

Where to use it

Local dev stacks, CI test environments, and Kubernetes production workloads.

Real use example

QA runs the exact same image SHA that production deployed yesterday.

Chapter 13: Configuration and Twelve-Factor App Patterns [Intermediate]

Chapter focus: Configuration and Twelve-Factor App Patterns** *(Level: Intermediate)

Store config in environment variables, not committed files. pydantic-settings loads and validates .env values at startup with clear errors. Separate dev, staging, and prod settings modules or use env-specific .env files locally only. Secrets belong in vaults-never log database URLs or API keys.

Code Reference:

```
class Settings(BaseSettings):
    model_config = SettingsConfigDict(env_file=".env")
    database_url: PostgresDsn
    jwt_secret: SecretStr

settings = Settings()
```

What it shows: SecretStr hides values in repr; invalid env fails fast at import time.

Topic guide

What it actually is

Externalized config lets the same artifact run in any environment by changing env vars.

When to use it

Apply from first deploy; refactor hardcoded constants into settings immediately.

Where to use it

Cloud deployments, Docker Compose, and GitHub Actions secret injection.

Real use example

Rotating JWT secret requires only env update-no code change or rebuild.

Track: Advanced

Chapter 14: Background Jobs with Celery [Advanced]

Chapter focus: Background Jobs with Celery** *(Level: Advanced)

Long-running tasks (email, reports, ML inference) belong outside HTTP request cycle. Celery workers consume tasks from Redis or RabbitMQ brokers asynchronously. Task retries with exponential backoff handle transient failures gracefully. Monitor queues with Flower; alert when backlog exceeds SLO thresholds.

Code Reference:

```
@celery_app.task(bind=True, max_retries=3)
def send_invoice(self, invoice_id: int):
    try:
        billing.send(invoice_id)
```

```
except SMTPException as exc:
    raise self.retry(exc=exc, countdown=60 * (2 ** self.request.retries))
```

What it shows: bind=True gives task instance for retry; countdown doubles each attempt.

Topic guide

What it actually is

Task queues decouple web requests from slow or unreliable background work.

When to use it

Use when operations exceed HTTP timeout or need scheduled/recurring execution.

Where to use it

Billing, email campaigns, data exports, and image processing pipelines.

Real use example

User gets 202 Accepted instantly while invoice PDF generates in a worker.

Chapter 15: OAuth2, JWT, and API Security [Advanced]

Chapter focus: OAuth2, JWT, and API Security** *(Level: Advanced)

OAuth2 authorization code flow delegates login to identity providers. JWT access tokens carry claims (sub, roles, exp) signed with HS256 or RS256. Validate exp and aud on every request; refresh tokens rotate with reuse detection. Rate limiting and CORS policies protect public endpoints from abuse.

Code Reference:

```
def get_current_user(token: str = Depends(oauth2_scheme)) -> User:
    payload = jwt.decode(token, settings.jwt_secret.get_secret_value(),
        algorithms=["HS256"])
    return user_service.get(payload["sub"])
```

What it shows: Dependency extracts bearer token and validates signature before handler executes.

Topic guide

What it actually is

Token-based auth secures stateless APIs consumed by SPAs and mobile apps.

When to use it

Implement on any API exposing user-specific or mutating operations.

Where to use it

SaaS dashboards, partner APIs, and microservice authorization.

Real use example

Compromised token expires in 15 minutes; refresh rotation blocks replay attacks.

Chapter 16: Observability with structlog and OpenTelemetry [Advanced]

Chapter focus: Observability with structlog and OpenTelemetry** *(Level: Advanced)

Structured logs (JSON) parse reliably in ELK, Loki, or CloudWatch. structlog binds context (request_id, user_id) per request via contextvars. OpenTelemetry auto-instruments FastAPI/Django and exports traces to OTLP collectors. Correlate logs and traces with shared trace_id for faster incident response.

Code Reference:

```
log.info("order_created",          order_id=order.id,          amount=str(order.total),
trace_id=trace.get_current_span().get_span_context().trace_id)
```

What it shows: Keyword arguments become JSON fields searchable in log aggregators.

Topic guide

What it actually is

Observability instrumentation makes production Python services debuggable at scale.

When to use it

Add before launch; retrofitting under outage pressure is painful and incomplete.

Where to use it

Production APIs, SLO dashboards, and on-call rotations.

Real use example

On-call filters Loki for trace_id from user report and sees exact DB slow query span.

Chapter 17: Performance Profiling and Optimization [Advanced]

Chapter focus: Performance Profiling and Optimization** *(Level: Advanced)

Measure before optimizing-py-spy samples production processes with low overhead. Profile ORM queries; N+1 patterns dominate Python web latency more than interpreter speed. Cache expensive reads with Redis TTL; invalidate on writes explicitly. Use orjson and async drivers for hot paths after profiling confirms bottlenecks.

Code Reference:

```
python -m scalene app/main.py # CPU + memory line profiler
# or: py-spy record -o profile.svg -- python -m uvicorn main:app
```

What it shows: Scalene highlights line-level CPU and memory; py-spy works on running prod processes.

Topic guide

What it actually is

Profiling identifies actual bottlenecks instead of guessing where time is spent.

When to use it

Profile when P95 latency exceeds SLO or CPU cost spikes after a release.

Where to use it

High-traffic APIs, batch jobs, and cost-sensitive cloud workloads.

Real use example

Adding selectinload cut query count from 201 to 3, dropping P95 from 2s to 180ms.

Chapter 18: Production Deployment and API Versioning [Advanced]

Chapter focus: Production Deployment and API Versioning *(Level: Advanced)**

Version APIs via URL (/v1/), header (Accept-Version), or subdomain-pick one strategy. Blue-green deploys swap traffic between identical environments with zero downtime. Health checks and graceful shutdown (SIGTERM handling) drain in-flight requests. Document deprecation timelines; sunset old versions with metrics proving low traffic.

Code Reference:

```
app = FastAPI()
v1 = APIRouter(prefix="/v1")
v2 = APIRouter(prefix="/v2")
app.include_router(v1)
app.include_router(v2)
```

What it shows: Separate routers isolate breaking changes; clients migrate to /v2 at their pace.

Topic guide

What it actually is

Deployment discipline and API versioning prevent breaking clients during rapid iteration.

When to use it

Apply versioning before external clients integrate; deploy with health checks from day one.

Where to use it

Public APIs, mobile app backends, and partner integrations.

Real use example

Mobile app v3 calls /v2 while legacy v1 serves 2% traffic until sunset date.